- LED and appropriate 220-ohm resistor
- TEMT6000 light sensor or other analog sensor
- Hookup wires
- Solderless breadboard

Using Libraries

To use a library in an Arduino sketch, we need to tell the compiler which library we want to use. There are two ways to do this. The first is to use a `#include` pre-processor directive at the beginning of our sketch to specify the library's file name, as follows:

```
#include <LibraryName.h>
```

`LibraryName.h` is the file name of the library that we want to use. In our last project example, we used the line `#include <LiquidCrystal.h>` to enable those functions that make it easier to use an LCD with our project. Pre-processor directives are instructions to the compiler that tell it to include the functions that we want to use at the time of compiling. They don't follow the normal syntax of statements and don't end with semicolons. We mention this here because every library listed later will need this line at the beginning of the sketch for it to work, but we will not break it out to its own section.

Another way to access libraries is through the Sketch menu and the option Import Library, and then select the library name from the menu. The programming environment will write the `#include` line to match the specific library for you. If we want to make use of a contributed library that is not on the list because it is not a part of the standard Arduino distribution, we need to properly install the library before we can access it. While this will not be necessary in any of the examples in this chapter, should you need to install a contributed library, you would need to create a folder named Libraries in the Arduino folder that contains our sketches, if there's not one already, and then copy the new library's folder into it. This library should contain at least two or three files, one ending in .h, as well as other files that could include example sketches or other documentation. Before this new library can be used, the Arduino programming environment will need to be restarted and then include or import the file as normal.

Creating an Instance

After including the library, one of the first things we will do is to either create a new instance of the library or initialize the library, depending on what the library does and how it is written. While this will be specific to each library, it is worth mentioning the basic mechanic for this now. Using our previous LCD example from the last chapter, we needed to create an instance of the LiquidCrystal library to begin working with it. This is done in the following line:

```
LiquidCrystal lcd(5, 6, 7, 8, 9, 10);
```

Theoretically it is possible to have multiple LCDs, servos, steppers, or other hardware in use by some libraries and so we could have multiple lines, like that just shown, with different instance names. Here we have created the instance named `lcd`, which is a variable name that links to the LiquidCrystal library and assigns the pin numbers to be used with it, according to the specifications of the library. Creating an instance of a library is generally only necessary when multiple instances can exist, as is the case here. If we had multiple instances of a library, for example we had multiple LCDs connected to the Arduino, then they would each need a unique name like `lcd1`, `lcd2`, and so on.